

PROCESSO SELETIVO TÉCNICO · JAVA

Este documento descreve o desafio, os requisitos obrigatórios e os critérios de avaliação. Leia tudo antes de começar — a entrega será avaliada exatamente pelos itens listados aqui.

Descrição do desafio

Você deve criar um **sistema de análise de dados** capaz de importar lotes de arquivos, processar e analisar os dados contidos neles e produzir relatórios automaticamente.

Estrutura dos dados

Existem **3 tipos de registros** dentro dos arquivos, cada um com seu próprio layout identificado por um prefixo numérico.

Dados do vendedor 001

```
001çCPFçNomeçSalário
```

Dados do cliente 002

```
002çCNPJçNomeçSegmento de Negócio
```

Dados de venda 003

```
003çID da VendaçItens da VendaçNome do Vendedor
```

Os itens da venda são delimitados por colchetes `[]`, separados por vírgula, com cada item no formato `ID-Quantidade-Preço`.

Exemplo de arquivo de entrada

```
001ç1234567891234çPedroç50000
001ç3245678865434çPauloç40000.99
002ç2345675434544345çJose da SilvaçRural
002ç2345675433444345çEduardo PereiraçRural
003ç10ç[1-10-100,2-30-2.50,3-40-3.10]çPedro
003ç08ç[1-34-10,2-33-1.50,3-40-0.10]çPaulo
```

Requisitos funcionais

Leitura de arquivos

- O sistema deve monitorar continuamente o diretório de entrada: `%HOMEPATH%/data/in`
- Apenas arquivos com extensão `.dat` devem ser processados
- Novos arquivos adicionados ao diretório devem ser detectados e processados **automaticamente**, sem necessidade de reiniciar a aplicação

Processamento

- O sistema deve ser capaz de processar **múltiplos arquivos** no diretório de entrada
- Linhas com formato inválido ou tipo desconhecido **não devem interromper** o processamento — devem ser registradas em log e ignoradas
- O sistema deve suportar **processamento concorrente** de múltiplos arquivos

Geração de relatório

Após processar cada arquivo, o sistema deve gerar um arquivo de saída no diretório:

`%HOMEPATH%/data/out`

Nome do arquivo de saída

```
{nome_do_arquivo_original}.done.dat
```

Conteúdo do arquivo de saída

```
Quantidade de clientes: {valor}
Quantidade de vendedores: {valor}
ID da venda mais cara: {valor}
Pior vendedor (menor volume de vendas): {nome}
```

Requisitos técnicos obrigatórios

Linguagem e build

Java 17

Maven ou Gradle

Execução via linha de comando

- O projeto deve compilar e executar via linha de comando sem configurações adicionais

Organização do código

- O candidato tem **liberdade para definir a estrutura de pacotes**, desde que siga **padrões de mercado** para projetos Java
- A estrutura adotada deve ser **consistente, coesa e justificada** no `README.md`

Uso de inteligência artificial

O uso de ferramentas de IA é **permitido e incentivado** como forma de ganhar eficiência e produtividade. O que avaliamos é o resultado e o seu domínio sobre ele — não quanto código você digitou.

- **Não entregue lixo gerado.** Código morto, abstrações desnecessárias, comentários óbvios, testes que não testam nada e dependências sem uso contam contra a avaliação
- **Justifique o uso.** Descreva no `README.md` onde a IA foi utilizada e com qual objetivo
- **Justifique as escolhas arquiteturais.** Cada decisão de estrutura, camada ou padrão deve ter uma razão que você saiba defender — inclusive as sugeridas pela IA
- **Revise o código antes de entregar.** Leia linha a linha, rode os testes e assuma o resultado como seu: você será questionado sobre qualquer trecho na etapa seguinte

EM RESUMO

Use IA para ir mais rápido, não para pensar menos. **Você é responsável por todo o código entregue.**

Testes

COBERTURA MÍNIMA

80% de cobertura verificada via **Codecov**. O build deve falhar abaixo desse limite.

- Utilizar **JUnit 5** para testes unitários
- Utilizar **Mockito** para mock de dependências
- Incluir testes de **integração** para o fluxo completo de leitura → processamento → escrita
- Testes devem cobrir ao menos:
 - Parsing correto de cada tipo de registro
 - Comportamento com linhas malformadas
 - Cálculo correto da venda mais cara
 - Identificação correta do pior vendedor
 - Geração correta do arquivo de saída

Integração contínua (CI/CD)

O projeto deve conter pipeline configurado no **CircleCI** (arquivo `.circleci/config.yml`), executando obrigatoriamente as seguintes etapas:

1. **Checkout** do código
2. **Build** do projeto
3. **Execução dos testes**
4. **Geração do relatório de cobertura** (JaCoCo)
5. **Envio do relatório ao Codecov**

Qualidade de código

- O projeto deve conter o arquivo de configuração do **Codecov** (`codecov.yml`) definindo o threshold mínimo de cobertura
- Badges de status do **CircleCI** e **Codecov** devem estar presentes no `README.md`

Estrutura esperada do repositório

```
analizador-de-dados/
|
├── .circleci/
|   └── config.yml                # Pipeline do CircleCI
|
├── src/
|   ├── main/
|   │   ├── java/...             # Código-fonte principal
|   │   └── resources/
|   │       └── application.properties # Configurações externalizadas
|   └── test/
|       ├── java/...             # Testes unitários e de integração
|       └── resources/
|           └── dados-teste.dat    # Arquivo .dat de exemplo para testes
|
├── codecov.yml                  # Configuração e threshold do Codecov
├── pom.xml ou build.gradle      # Gerenciador de dependências
└── README.md                    # Documentação com badges e instruções
```

README mínimo esperado

O `README.md` deve conter:

- ☐ Badge de status do **CircleCI** (build passing/failing)
- ☐ Badge de cobertura do **Codecov**
- ☐ Descrição resumida do projeto
- ☐ Instruções de **como compilar e executar**
- ☐ Instruções de **como executar os testes**
- ☐ Descrição das **decisões de arquitetura** adotadas, incluindo a justificativa para a estrutura de pacotes escolhida
- ☐ Declaração de **uso de IA**: onde foi utilizada, com qual objetivo e como o código foi revisado

Critérios de avaliação

Critério	Descrição
Clean Code	Legibilidade, nomenclatura clara, ausência de código morto
Simplicidade	Solução objetiva, sem complexidade desnecessária
Lógica	Corretude dos cálculos e análises produzidas
Separação de responsabilidades	Camadas bem definidas, coesas e com baixo acoplamento
Flexibilidade / Extensibilidade	Facilidade para adicionar novos tipos de registro ou análises
Escalabilidade / Performance	Eficiência no processamento com uso adequado de Streams e I/O
Cobertura de testes	Mínimo de 80% verificado via Codecov
Pipeline CI	CircleCI configurado e funcional com todas as etapas
Documentação	README claro com badges, instruções e decisões técnicas

Entrega

- Enviar o **link do repositório público no GitHub**
- O repositório deve estar com o **CircleCI integrado** e os **badges visíveis no README**
- O **Codecov** deve estar configurado e exibindo a cobertura atualizada

ATENÇÃO

Projetos sem pipeline funcional ou sem testes automatizados **não serão avaliados.**